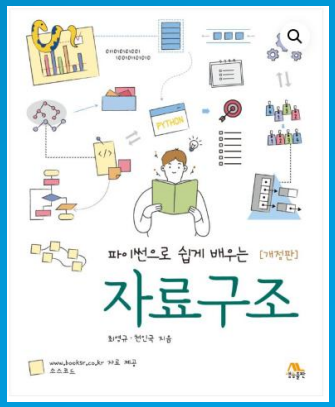


CHAPTER 07

정렬과 탐색

7장. 정렬과 탐색



7.1 정렬이란?

7.2 기본적인 정렬 알고리즘

7.3 정렬 응용: 집합 다시보기

7.4 탐색과 맵 구조

7.5 기본적인 탐색 알고리즘

7.6 고급 탐색 구조: 해싱

7.7 해싱의 오버플로 해결 방법

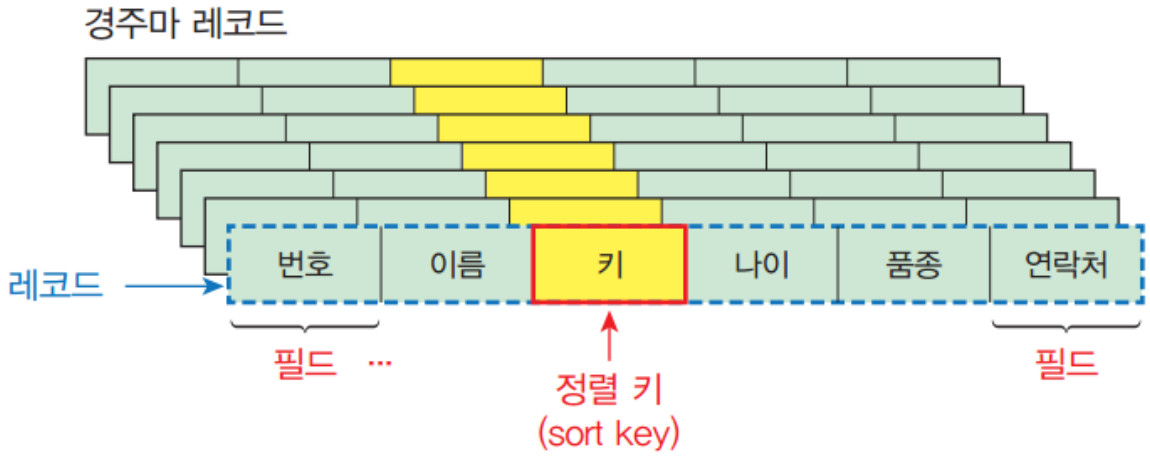
7.1 정렬이란?

- 데이터를 순서대로 재배열하는 것
  - 가장 기본적이고 중요한 알고리즘
  - 비교할 수 있는 모든 속성들은 정렬의 기준이 될 수 있다
  - 오름차순(ascending order)과 내림차순(descending order)



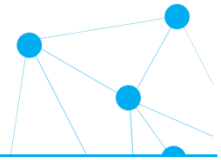
용어들

- 레코드: 정렬시켜야 될 대상
  - 여러 개의 필드(field)로 이루어짐
  - 정렬 키(sort key): 정렬의 기준이 되는 필드

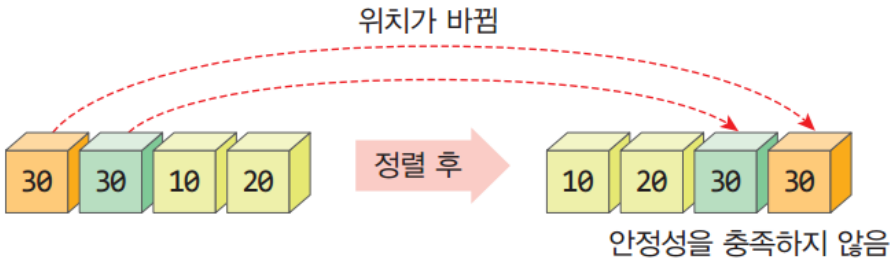


- 정렬이란 레코드들을 키(key)의 순서로 재배열하는 것

정렬 알고리즘 종류

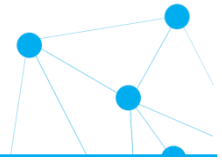


- 복잡도와 효율성에 따른 분류
  - 단순하지만 비효율적: 삽입, 선택, 버블 정렬 등
  - 복잡하지만 효율적: 퀵, 힙, 병합, 기수 정렬, 팀 등
- 킷값의 비교를 사용하지 않는 정렬: 기수, 카운팅 정렬 등
- 정렬 장소에 따른 분류
  - 내부(internal) 정렬: 모든 데이터가 메인 메모리
  - 외부(external) 정렬: 외부 기억 장치에 대부분의 레코드
- 안정성(stability)



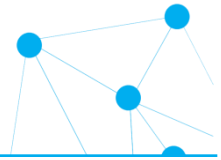
- 제자리 정렬(in-place sorting)
  - 추가 메모리 사용 여부

7.2 기본적인 정렬 알고리즘



- 선택 정렬(selection sort)
- 삽입 정렬(insertion sort)
- 버블 정렬(bubble sort)

선택 정렬(selection sort)



- 오른쪽 리스트에 서 가장 작은 숫자를 선택하여 왼쪽 리스트의 맨 뒤로 이동하는 작업을 반복

정렬 된(왼쪽) 리스트	정렬 안 된(오른쪽) 리스트	설명
[ ]	[5,3,8,4,9,1,6,2,7]	초기상태
[1]	[5,3,8,4,9,6,2,7]	1선택 및 이동
[1,2]	[5,3,8,4,9,6,7]	2선택 및 이동
[1,2,3]	[5,8,4,9,6,7]	3선택 및 이동
…	…	4~8 선택 및 이동
[1,2,3,4,5,6,7,8,9]	[ ]	9선택 및 이동

선택 정렬 알고리즘

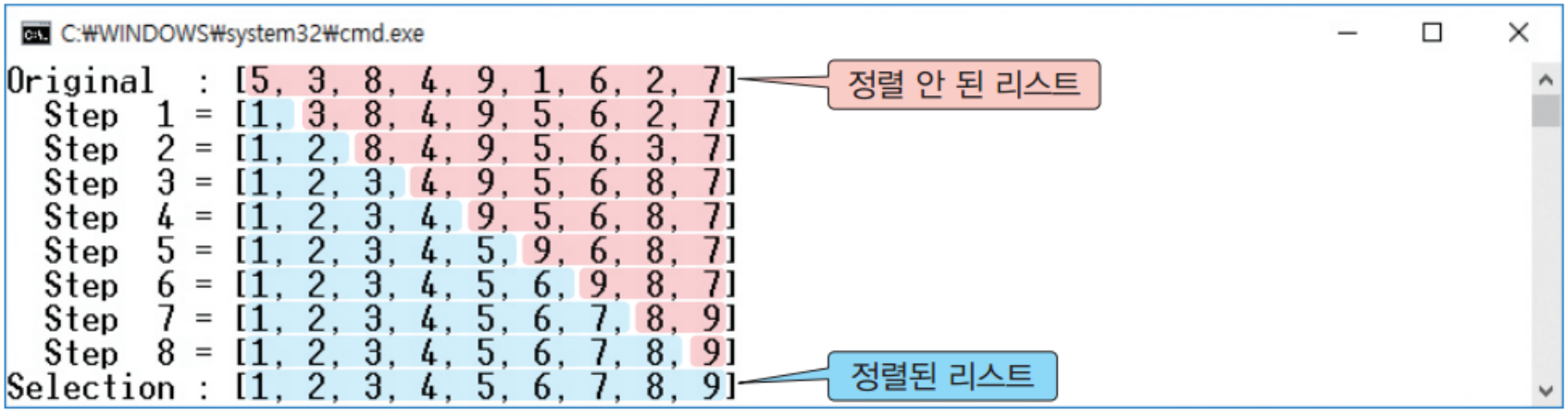



```
def selection_sort(A) :
    n = len(A)
    for i in range(n-1) :
        least = i;
        for j in range(i+1, n) :
            if (A[j]<A[least]) :
                least = j
        A[i], A[least] = A[least], A[i]
        printStep(A, i + 1);
```

- 시간 복잡도: $(n-1)+(n-2)+\cdots+1=n(n-1)/2=O(n^2)$
- 효율적이지 않음, 안정성 만족 않음
- 알고리즘이 간단, 자료 이동 횟수가 미리 결정됨

테스트 프로그램

```
data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("Original  : ", data)
selection_sort(data)
print("Selection : ", data)
```



삽입 정렬(insertion sort)

- 정렬되어 있는 부분에 새로운 레코드를 올바른 위치에 삽입하는 과정 반복



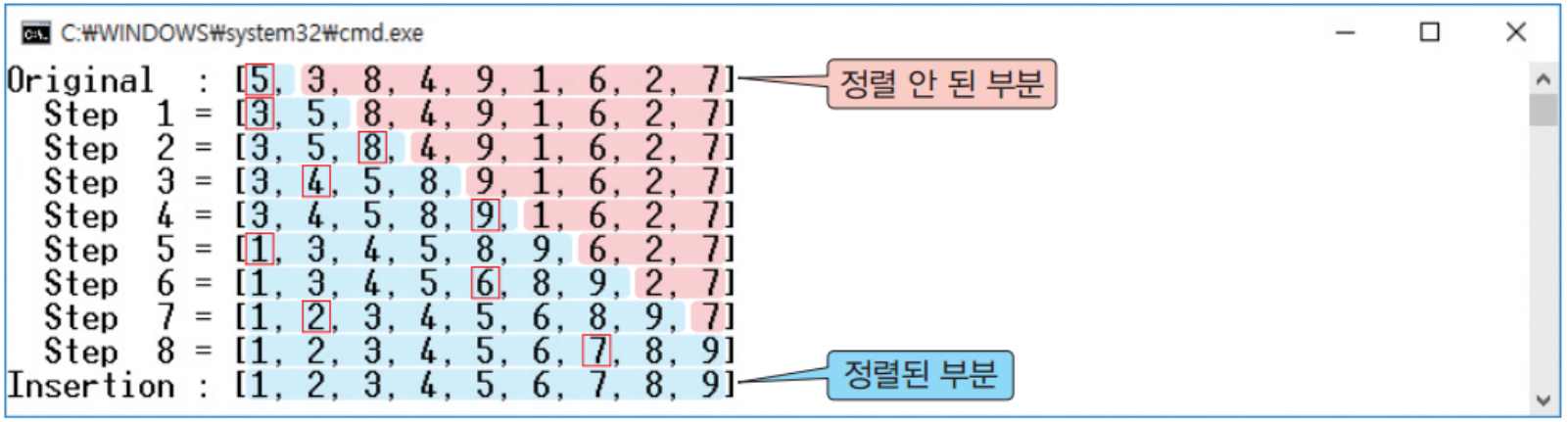


- 숫자를 끼워 넣으려면 많은 이동 발생
  - 교환하는 방식?

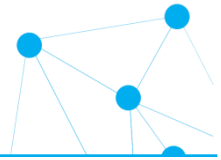
삽입 정렬 알고리즘




```
def insertion_sort(A) :
    n = len(A)
    for i in range(1, n) :
        key = A[i]
        j = i-1
        while j>=0 and A[j] > key :
            A[j + 1] = A[j]
            j -= 1
        A[j + 1] = key
        printStep(A, i)
```

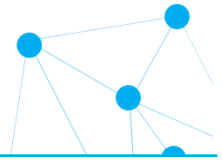


삽입 정렬 분석



- 복잡도 분석
  - 최선의 경우 O(n) : 이미 정렬되어 있는 경우: 비교: *n-1* 번
  - 최악의 경우 $O(n^2)$: 역순으로 정렬되어 있는 경우
    - 모든 단계에서 앞에 놓인 자료 전부 이동
    - 비교: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$
    - 이동: $\frac{n(n-1)}{2} + 2(n-1) = O(n^2)$
  - 평균의 경우 $O(n^2)$
- 특징
  - 많은 이동 필요 → 레코드가 큰 경우 불리
  - 안정된 정렬방법
  - 대부분 정렬되어 있으면 매우 효율적

버블 정렬 (bubble sort)



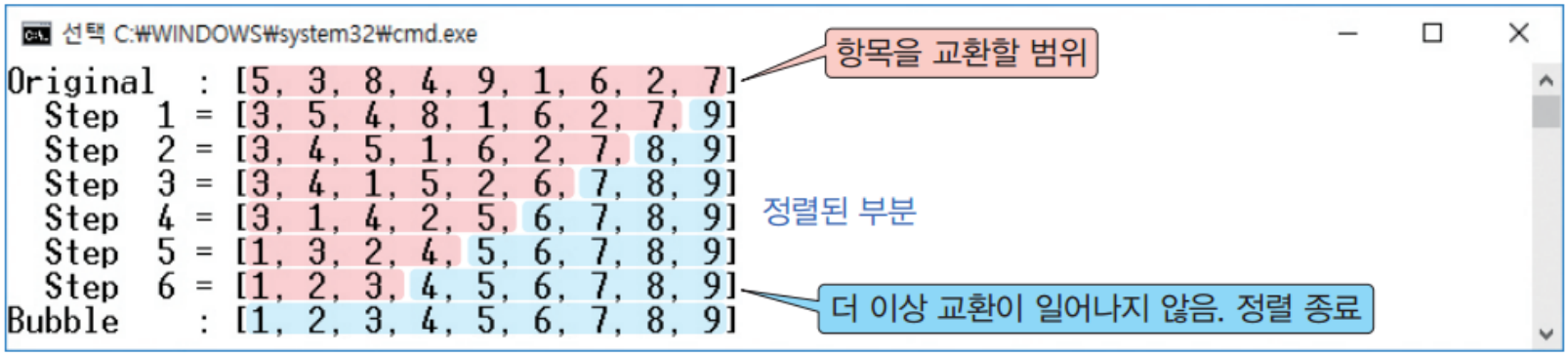
- 기본 전략
  - 인접한 2개의 레코드를 비교하여 순서대로 서로 교환
  - 비교-교환 과정을 리스트의 전체에 수행(스캔)
    - 한번의 스캔이 완료되면 리스트의 오른쪽 끝에 가장 큰 레코드
  - 끝으로 이동한 레코드를 제외하고 다시 스캔 반복

버블 정렬 알고리즘

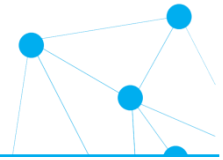



```
def bubble_sort(A) :
    n = len(A)
    for i in range(n-1, 0, -1) :
        bChanged = False
        for j in range (i) :
            if (A[j]>A[j+1]) :
                A[j], A[j+1] = A[j+1], A[j]
                bChanged = True

        if not bChanged: break;
        printStep(A, n - i);
```



버블 정렬 분석

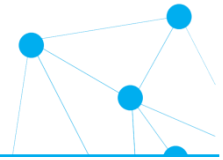


- 비교 횟수(최상, 평균, 최악의 경우 모두 일정)

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$

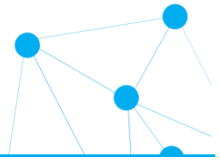
- 이동 횟수
  - 역순으로 정렬된 경우(최악): 이동 횟수 = 3 * 비교 횟수
  - 이미 정렬된 경우(최선의 경우) : 이동 횟수 = 0
  - 평균의 경우 : $O(n^2)$

- 레코드의 이동 과다
  - 이동연산은 비교연산 보다 더 많은 시간이 소요됨

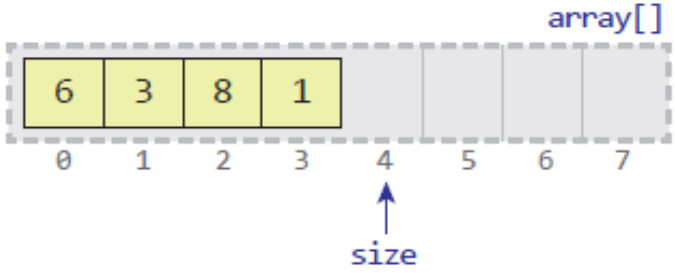


- p244
- 중간 점검 문제 풀이

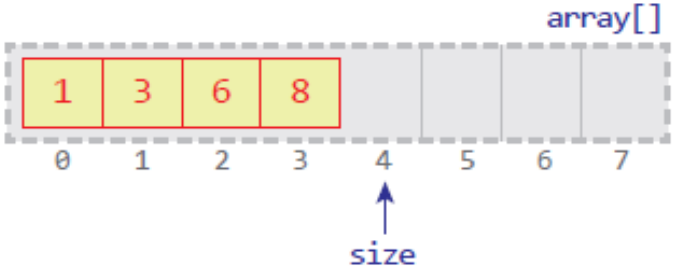
7.3 정렬 응용: 집합 다시 보기



- 배열에 집합의 원소를 저장하는 두 가지 방법



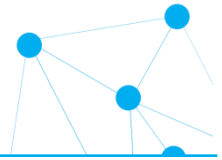
(a) 배열 요소를 정렬하지 않고 저장한 경우



(b) 배열 요소를 오름차순으로 정렬하여 저장한 경우

- 집합의 원소들을 배열에 정렬하여 관리
  - 연산들의 성능에 차이가 발생
  - 삽입 연산은 더 복잡해 짐
  - 원소가 집합에 포함되어 있는지 검사 → 효율적 구현 가능
    - 순차 탐색 → 이진 탐색
  - 집합의 비교나 합집합, 차집합, 교집합 → 효율적 구현 가능

삽입 연산



- 삽입 과정
  - 중복 검사 : O(n)
  - 삽입할 위치 찾기 : O(n)
  - 해당 위치에 삽입 : O(n)
- ArraySet 클래스의 insert()연산 구현 예: O(n)

```
def insert(self, e) :
    if self.contains(e) or self.isFull() :
        return
```

이미 있는 원소이거나 집합이 포화상태이면 삽입할 수 없음

```
self.array[self.size] = e
self.size += 1
```

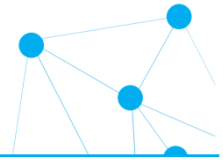
일단 정렬된 배열의 맨 뒤에 추가함

```
for i in range(self.size-1, 0, -1) :
    if self.array[i-1] <= self.array[i] :
        break
    self.array[i-1], self.array[i] = self.array[i], self.array[i-1]
```

맨 뒤에서부터 앞의 원소가 더 크면 원소를 교환. 이 과정을 작거나 같은 원소가 나올 때까지 반복

원소 교환

집합의 비교 연산: __eq__



- 비교 방법
  - 두 집합의 원소의 개수가 같아야 함
  - 정렬되어 있으므로 순서대로 같은 원소를 가져야 함

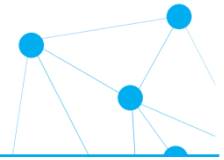
- $O(n^2)$구현 → $O(n)$으로 개선

```
def __eq__( self, setB ):
    if self.size != setB.size :
        return False

    for i in range(self.size) :
        if not setB.contains(self.array[i]) :
            return False
    return True
```

```
if self.array[i] != setB.array[i] :
```

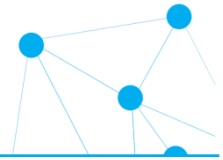
합집합/교집합/차집합



- 합집합 연산 방법
  - 가장 작은 원소들부터 비교하여 더 작은 원소를 새로운 집합에 넣고 그 집합의 인덱스를 증가시킴.
  - 만약 두 집합의 현재 원소가 같으면 하나만을 넣음. 인덱스는 모두 증가시킴.
  - 한쪽 집합이 모두 처리되면 나머지 집합의 남은 모든 원소를 순서대로 새 집합에 넣음

- 시간 복잡도:$O(n^2)$ ➔ $O(n)$으로 개선
- 교집합과 차집합도 동일한 방법 적용 가능

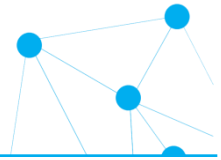
복잡도 비교



- 정렬되지 않은 리스트와 정렬된 리스트로 구현한 집합에서의 복잡도 비교

집합의 연산	정렬되지 않은 리스트	정렬된 리스트
`insert(e)`	$O(n)$	$O(n)$
`__eq__(setB)`	$O(n^2)$	$O(n)$
`union(setB)`	$O(n^2)$	$O(n)$
`intersect(setB)`	$O(n^2)$	$O(n)$
`difference(setB)`	$O(n^2)$	$O(n)$
`contains(e)`	$O(n)$, 순차탐색	$O(\log n)$, 이진탐색

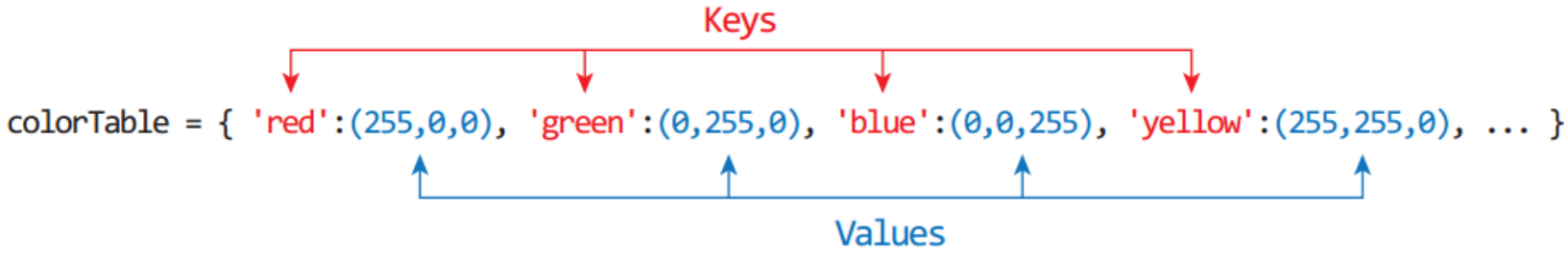
7.4 탐색과 맵 구조



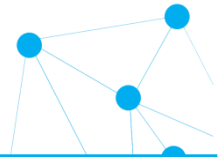
- 탐색이란?
  - 테이블에서 원하는 탐색키를 가진 레코드를 찾는 작업
- 맵(map) 또는 딕셔너리 (dictionary)
  - 탐색을 위한 자료구조
  - 엔트리(entry), 또는 키를 가진 레코드 (keyed record)의 집합



- 파이썬의 딕셔너리



맵의 추상 자료형



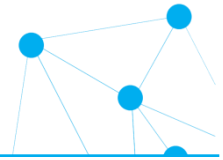
데이터: 키를 가진 레코드(엔트리)의 집합

연산

- search(key): 탐색키 key를 가진 레코드를 찾아 반환한다.
- insert(entry): 주어진 entry를 맵에 삽입한다.
- delete(key): 탐색키 key를 가진 레코드를 찾아 삭제한다.

- 맵을 구현하는 방법
  - 리스트 이용: 정렬 / 비정렬
  - 이진 탐색 트리 이용 (9장)
  - 해싱 구조 이용

7.5 기본적인 탐색 알고리즘



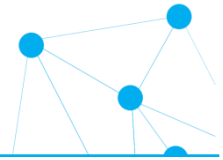
- 순차 탐색(sequential search)
- 이진 탐색(binary search)
- 보간 탐색(interpolation search)

순차 탐색(sequential search)

- 정렬되지 않은 배열에 적용 가능
  - 정렬되지 않은 배열을 처음부터 마지막까지 하나씩 검사
  - 가장 간단하고 직접적인 탐색 방법
  - 평균 비교 횟수: (n + 1)/2번 비교 (최악의 경우: n번)



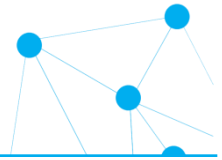
순차 탐색 알고리즘



```
def sequential_search(A, key, low, high) :      # 순차탐색
    for i in range(low, high+1) :               # i : low, low+1, ... high
        if A[i].key == key :                    # 탐색 성공하면
            return i                            # 인덱스 반환
    return None                                 # 탐색에 실패하면 None 반환
```

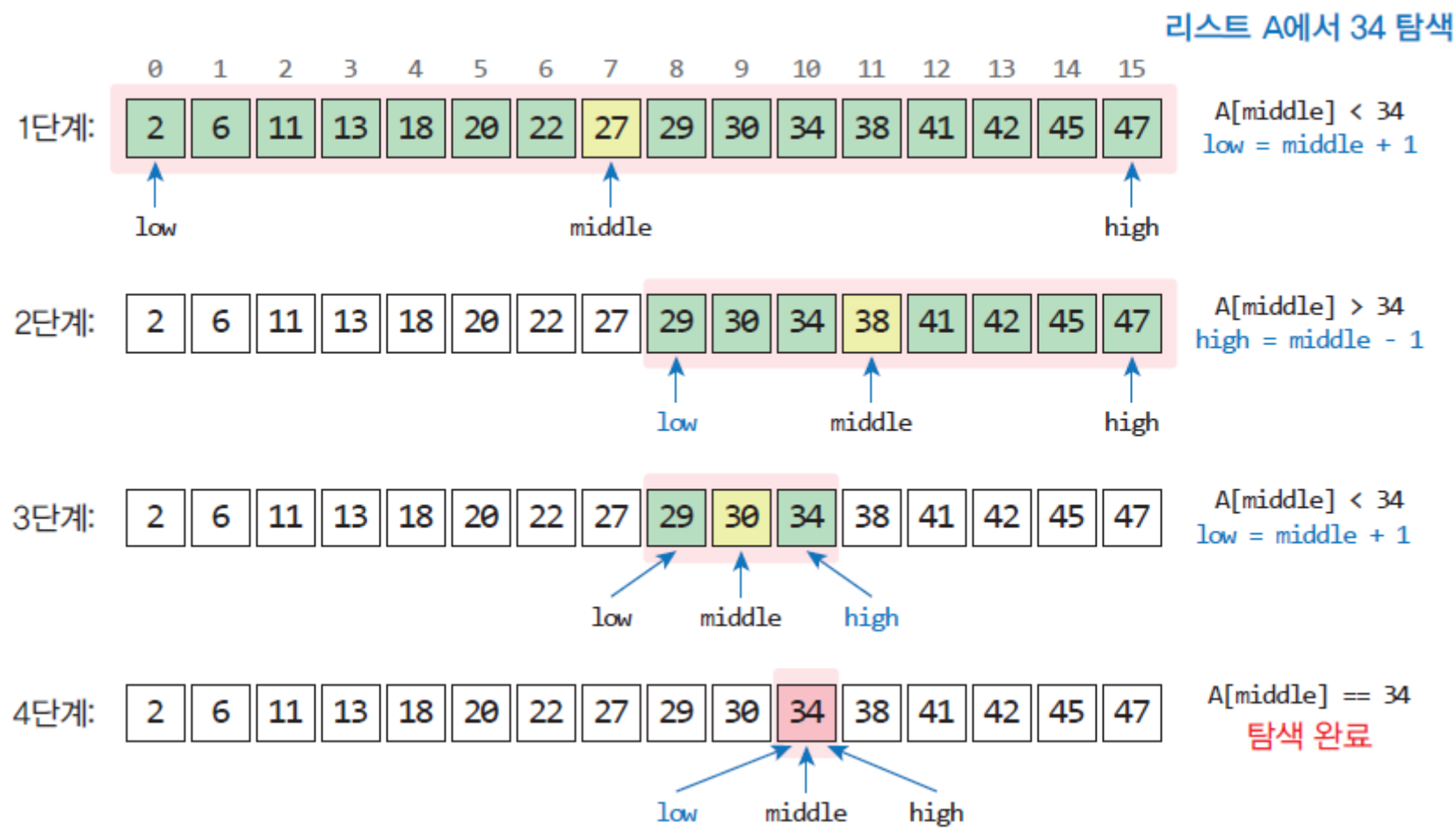
- 시간 복잡도: $O(n)$

이진 탐색(binary search)

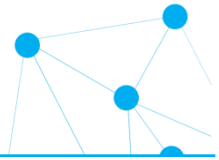


- 정렬된 배열의 탐색에 적합
  - 배열의 중앙에 있는 값을 조사하여 찾고자 하는 항목이 왼쪽 또는 오른쪽 부분 배열에 있는지를 알아내어 탐색의 범위를 반으로 줄여가며 탐색 진행
  - 예) 사전에서 단어 찾기
- (예) 10억 명중에서 특정한 이름 탐색
  - 이진탐색 : 단지 30번의 비교 필요
  - 순차 탐색 : 평균 5억 번의 비교 필요

이진 탐색 과정


리스트 A에서 34 탐색
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
1단계:
2 6 11 13 18 20 22 27 29 30 34 38 41 42 45 47
low
middle
high
A[middle] < 34
low = middle + 1
2단계:
2 6 11 13 18 20 22 27 29 30 34 38 41 42 45 47
low
middle
high
A[middle] > 34
high = middle - 1
3단계:
2 6 11 13 18 20 22 27 29 30 34 38 41 42 45 47
low
middle
high
A[middle] < 34
low = middle + 1
4단계:
2 6 11 13 18 20 22 27 29 30 34 38 41 42 45 47
low
middle
high
A[middle] == 34
탐색 완료

이진 탐색 알고리즘(순환 구조)



```
def binary_search(A, key, low, high) :
    if (low > high) :
        return -1

    middle = (low + high) // 2

    if (key == A[middle]) :
        return middle
    elif (key < A[middle]) :
        return binary_search(A, key, low, middle - 1)
    else :
        return binary_search(A, key, middle + 1, high)
```

순환의 종료 조건: 탐색할 항목이 하나도 없으면 탐색 실패. -1 반환

중앙 위치 middle 계산

탐색 성공. middle 반환

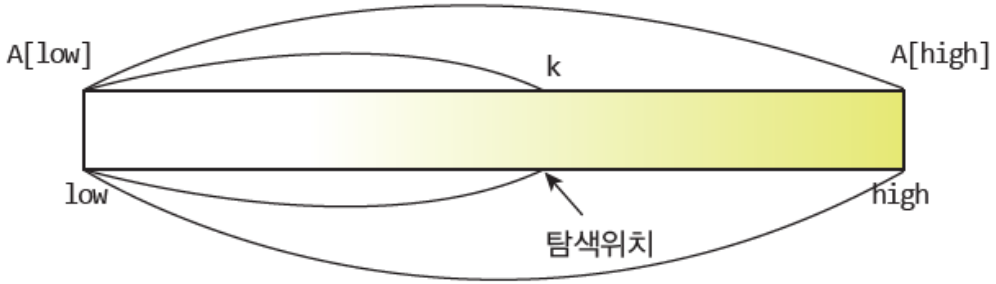
key가 A[middle]보다 작으면, 찾는 값은 왼쪽에 있음. 탐색 범위 low~middle-1을 순환 호출

A[middle]이 킷값보다 작으면, 찾는 값은 오른쪽에 있음. 탐색 범위 middle+1~high를 순환 호출

- 시간 복잡도: $O(logn)$

보간 탐색(interpolation search)

- 탐색키가 존재 할 위치를 예측하여 탐색
  - 예) 사전이나 전화번호부를 탐색할 때 'ㄱ' 시작 단어는 앞에서 'ㅎ' 시작 단어는 뒤에서 찾음
- 리스트를 불균등하게 분할하여 탐색
  - 탐색 값과 위치는 비례한다는 가정

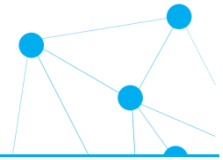


$$\text{탐색위치} = low + (high - low) \cdot \frac{k - A[low]}{A[high] - A[low]}$$

```
middle = int(low + (high-low) * (key-A[low].key) / (A[high].key-A[low].key))
```

- 중간 점검 문제
- p 257
- 1~5 까지 풀어보기

7.6 고급 탐색 구조: 해싱

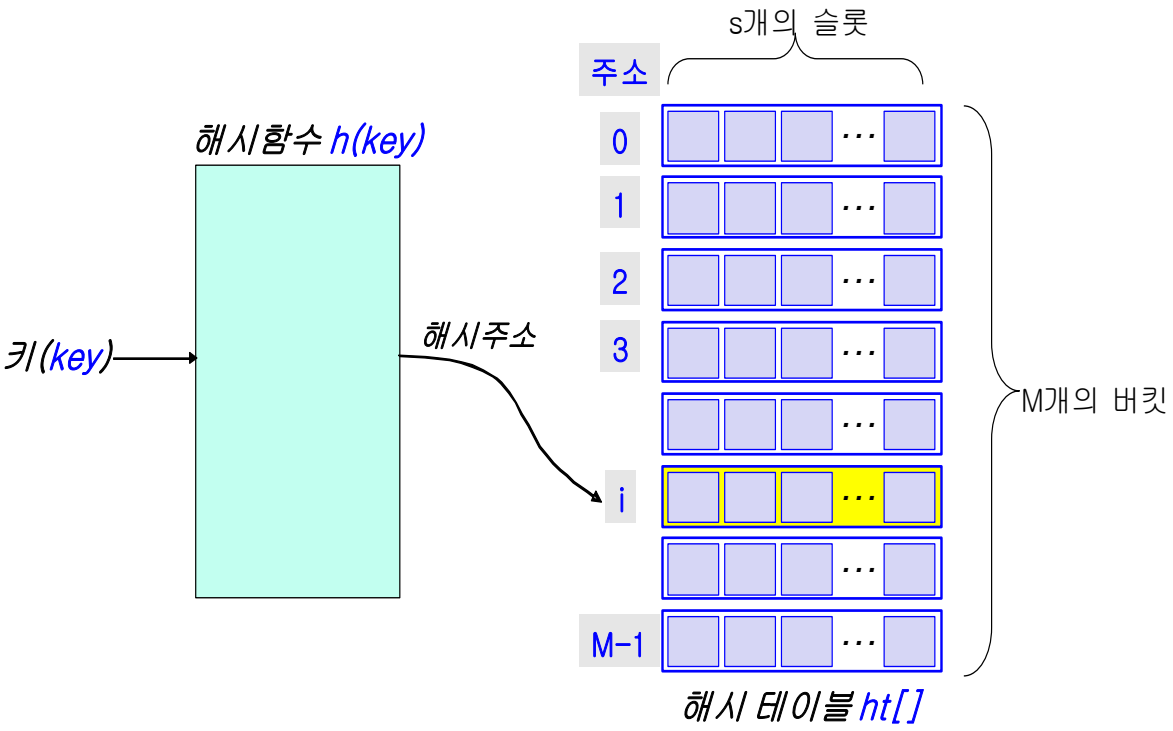


- 해싱(hashing) 이란?
  - 키 값에 대한 산술적 연산에 의해 레코드가 저장될 위치(주소)를 계산
  - 삽입/삭제/탐색 연산은 이 주소에서 이루어짐
    - 아파트의 세대별 우편함과 유사함
  - 이상적인 해싱에서 삽입/삭제/탐색 연산의 복잡도는 O(1)

- 실제의 해싱
  - 탐색키가 실수이거나 문자열, 굉장히 큰 정수라면? 메모리 부족!
  - 탐색키를 작은 정수로 대응시키는 함수 필요 → 해시 함수
  - 성능이 O(1)보다 떨어짐.

해싱의 구조

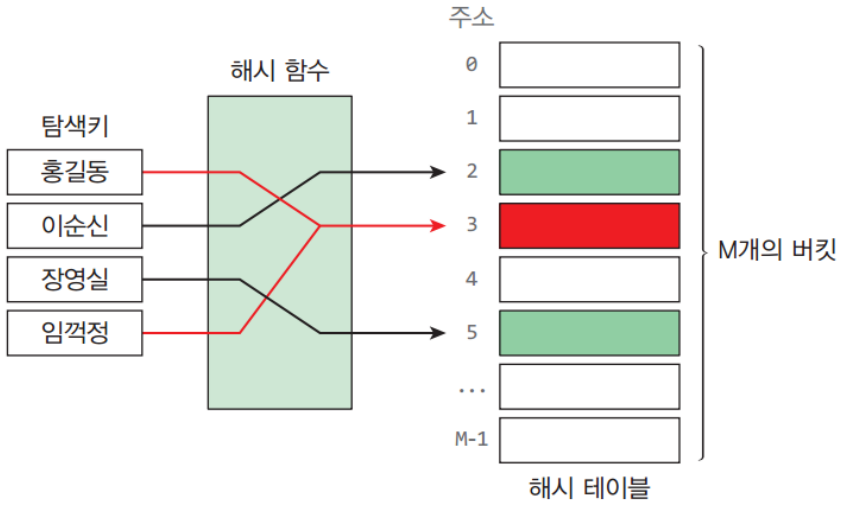
- 해시 테이블, 버킷, 슬롯
- 해시 함수(hash function)
  - 탐색키를 입력 받아 해시 주소(hash address) 생성



충돌과 오버플로

- 충돌
  - 서로 다른 키 가 해시 함수에 의해 같은 주소로 계산되는 상황
  - 예)

h(홍길동)⇒3, h(이순신)⇒2, h(장영실)⇒5, h(임꺽정)⇒3



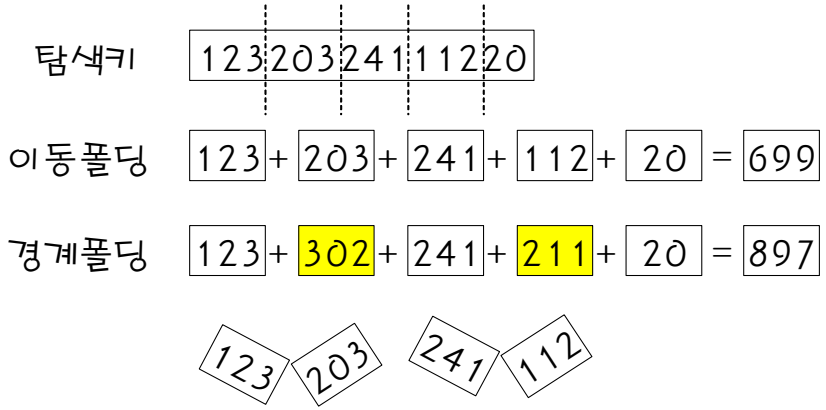
- 오버플로(overflow)
  - 충돌이 버킷의 슬롯 수보다 많이 발생하는 경우

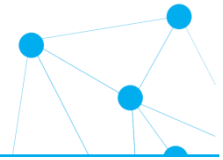
해시 함수

- 좋은 해시 함수의 조건
  - 충돌이 적어야 한다
  - 함수 값이 테이블의 주소 영역 내에서 고르게 분포되어야 한다
  - 계산이 빨라야 한다

- 제산 함수
  - h(k)=k mod M
  - 해시 테이블의 크기 M은 소수(prime number) 선택

- 폴딩 함수

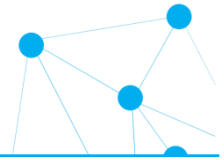




- 중간 제곱 함수
  - 탐색키를 제곱한 다음, 중간의 몇 비트를 취해서 해시 주소 생성
- 비트 추출 함수
  - 키를 이진수로 간주. 임의의 위치의 k개의 비트를 사용
- 숫자 분석 방법
  - 키에서 편중되지 않는 수들을 테이블의 크기에 적합하게 조합
- 탐색키가 문자열인 경우

```
def hashFn(key) :
    sum = 0
    for c in key :                  # 문자열의 모든 문자에 대해
        sum = sum +  ord(c)         # 그 문자의 아스키 코드 값을 sum에 더함
    return sum % M
```

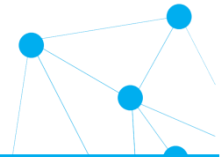
7.7 해싱의 오버플로 해결 방법



- 개방 주소법(open addressing)
  - 오버플로가 일어나면 테이블의 다른 위치에 저장하는 방법
  - 선형 조사법, 이차 조사법, 이중 해싱법 등

- 체이닝(chaining)
  - 하나의 주소에 여러 항목을 저장하도록 테이블의 구조 변경

선형 조사법

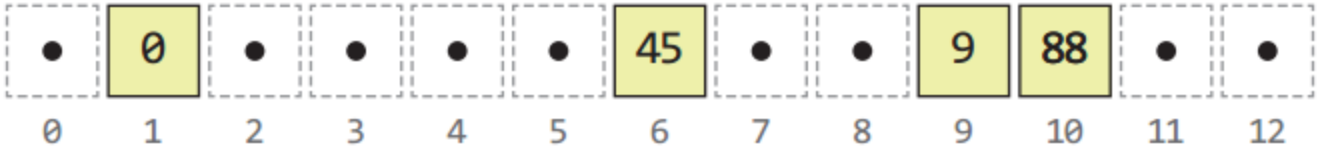


- 충돌이 일어나면 해시 테이블의 다음 위치에 저장
  - 다음 항목을 순서대로 조사: h(k), h(k)+1, h(k)+2,...
  - 빈 곳이 있으면 저장

- 예) 45, 27, 88, 9, 71, 60, 46, 38, 24 저장 과정
  - 테이블의 크기 M=13, $h(k) = k \% M$인 경우

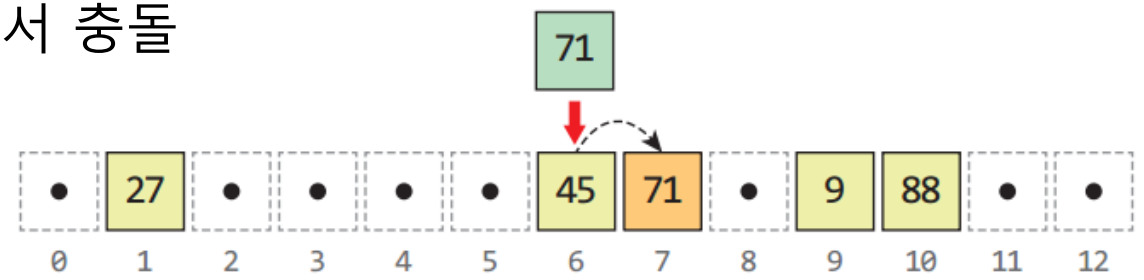
key	45	27	88	9	71	60	46	38	24
h(key)	6	1	10	9	6	8	7	12	11

① 45, 27, 88, 9 까지의 삽입



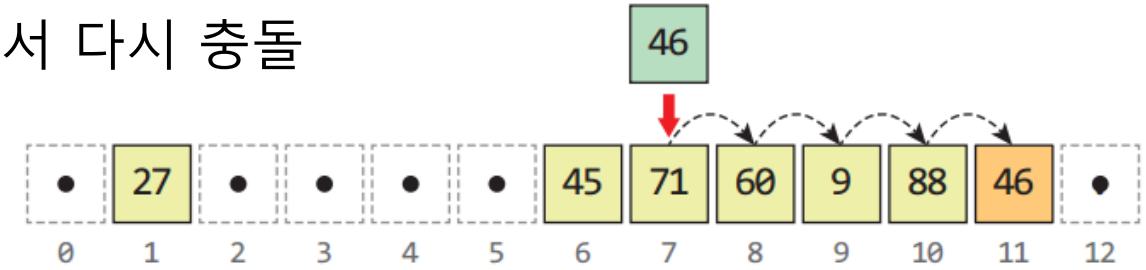
삽입 연산

② 71의 삽입에서 충돌

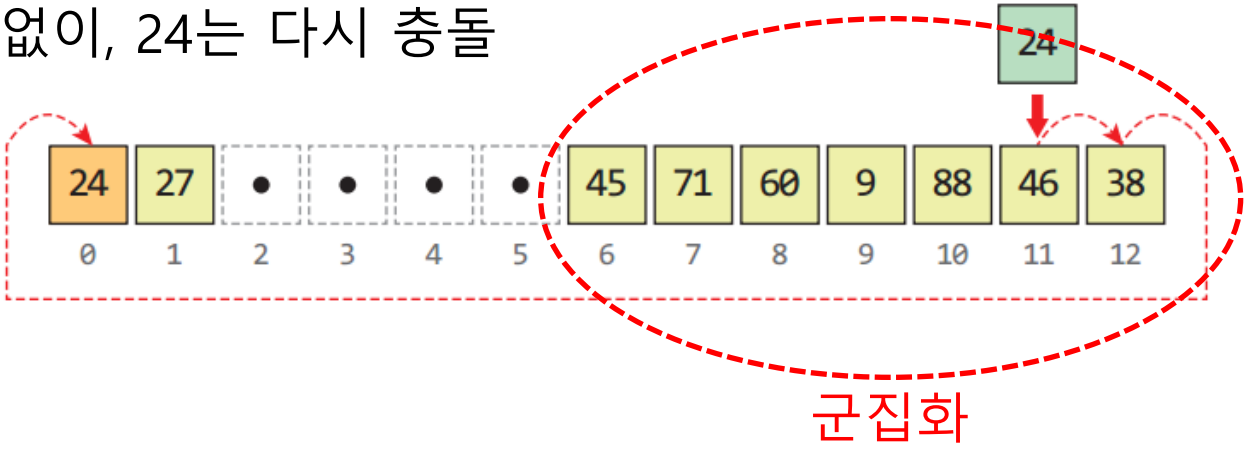


③ 60의 삽입

④ 46의 삽입에서 다시 충돌

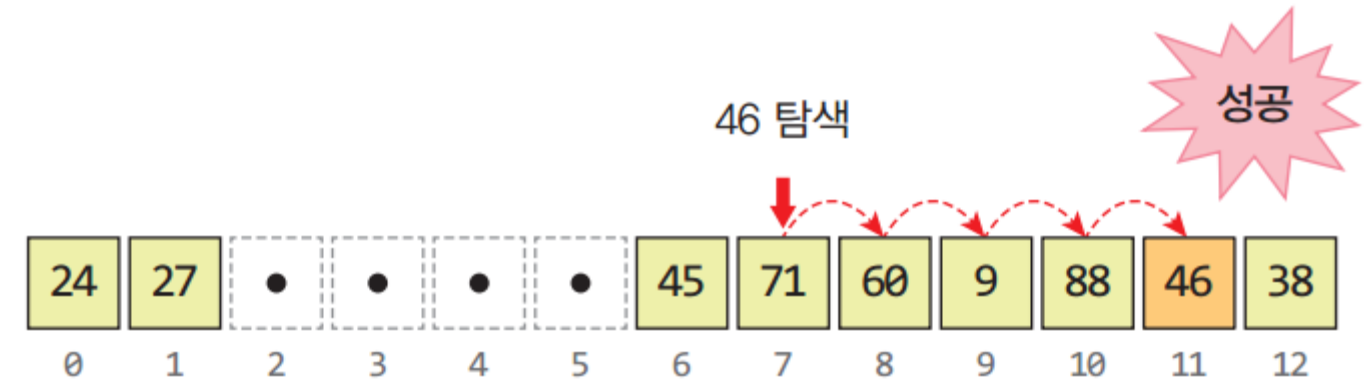


⑤ 38은 충돌 없이, 24는 다시 충돌

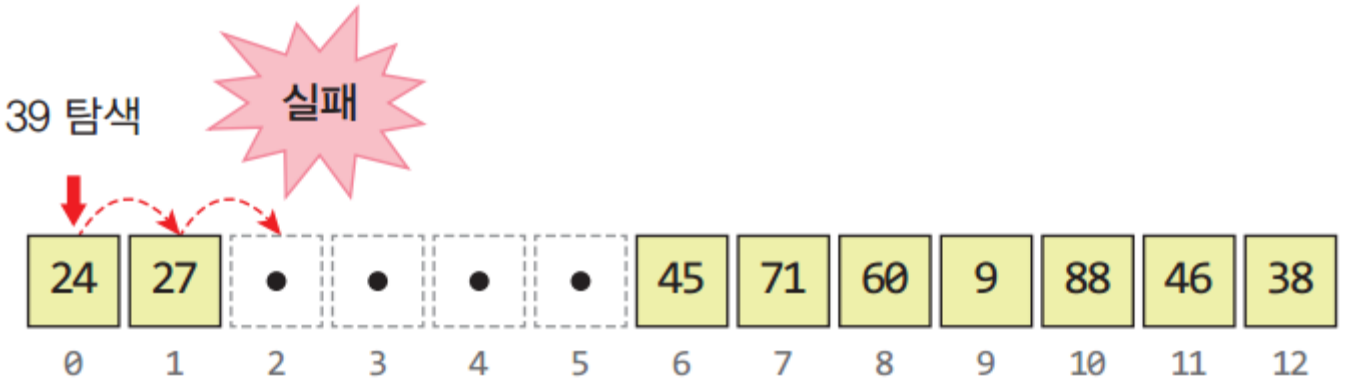


탐색 연산

- 46 탐색



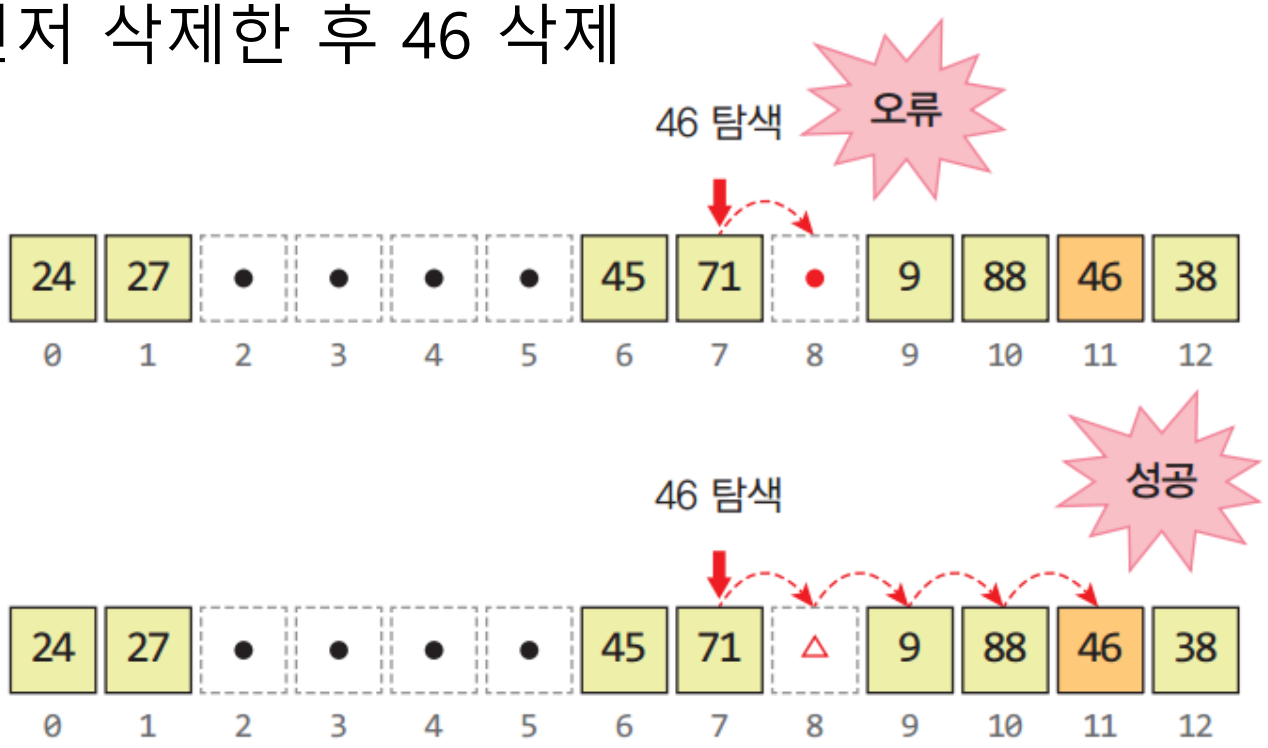
- 39 탐색



- 코드 7.13

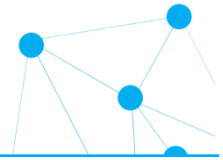
삭제 연산

- 60을 먼저 삭제한 후 46 삭제



- 코드 7.14
- 빈 버킷을 두 가지로 분류해야 함.
  - 한 번도 사용하지 않은 것
  - 사용 되었다가 삭제되어 현재 비어있는 것

테스트 프로그램



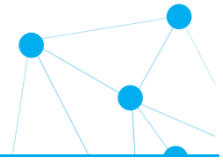
```
data = [45, 27, 88, 9, 71, 60, 46, 38, 26]
for d in data :
    print( "h(%2d)=%2d"%(d,hashFn(d)), end=' ')
    insert(d)
    print(table)

print("46탐색-->", search(46))
print("39탐색-->", search(39))

print("60삭제-->", end='')
delete(60)
print(table)
print("46삭제-->", end='')
delete(46)
print(table)
```

```
C:₩WINDOWS₩system32₩cmd.exe
h(45)= 6 [None, None, None, None, None, None, 45, None, None, None, None, None, None]
h(27)= 1 [None, 27, None, None, None, None, 45, None, None, None, None, None, None]
h(88)=10 [None, 27, None, None, None, None, 45, None, None, None, 88, None, None]
h( 9)= 9 [None, 27, None, None, None, None, 45, None, None, 9, 88, None, None]
h(71)= 6 [None, 27, None, None, None, None, 45, 71, None, 9, 88, None, None]
h(60)= 8 [None, 27, None, None, None, None, 45, 71, 60, 9, 88, None, None]
h(46)= 7 [None, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, None]
h(38)=12 [None, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
h(24)=11 [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
46탐색--> 46
39탐색--> None
60삭제-->[24, 27, None, None, None, None, 45, 71, -1, 9, 88, 46, 38]
46삭제-->[24, 27, None, None, None, None, 45, 71, -1, 9, 88, -1, 38]
```

군집화 완화 방법



- 이차 조사법(quadratic probing)

$$(h(k)+i*i)\ \%\ M \quad \text{for} \quad i=0,1,\cdots,M-1$$

- 이중 해싱법(double hashing)
  - 재해싱(rehashing)
  - 충돌이 발생하면, 다른 해시 함수를 이용해 다음 위치 계산

체이닝

- 하나의 버킷에 여러 개의 레코드를 저장
  - 예) h(k)=k%7 을 이용해 8, 1, 9, 6, 13 을 삽입

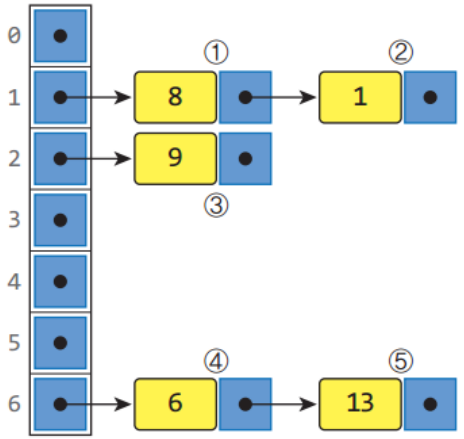
① 8 저장 : h(8) = 8 % 7 = 1 ⇒ 저장

② 1 저장 : h(1) = 1 % 7 = 1 ⇒ 충돌 ⇒ 새로운 노드 생성 및 저장

③ 9 저장 : h(9) = 9 % 7 = 2 ⇒ 저장

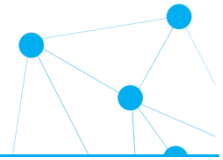
④ 6 저장 : h(6) = 6 % 7 = 6 ⇒ 저장

⑤ 13저장 : h(13) = 13 % 7 = 6 ⇒ 충돌 ⇒ 새로운 노드 생성 및 저장



- 여러 레코드 저장 방법?
  - 연결 리스트나 파이썬의 리스트 등
- 특징
  - 오버플로가 발생 → 해당 버킷의 연결 리스트만 처리
  - 필요한 만큼의 메모리만 사용 → 공간적으로 효율적
- 구현: 코드 7.16

탐색 방법들의 성능 비교



- 적재 밀도(loading density) 또는 적재 비율
  - 저장되는 항목의 개수 n과 해시 테이블의 크기 M의 비율

$$\alpha = \frac{\text{저장된 항목의 개수}}{\text{해싱테이블의 버킷의 개수}} = \frac{n}{M}$$

- 다양한 탐색 방법의 성능 비교

탐색방법		탐색	삽입	삭제
순차탐색		$O(n)$	$O(1)$	$O(n)$
이진탐색		$O(\log_2 n)$	$O(n)$	$O(n)$
이진탐색트리	균형트리	$O(\log_2 n)$	$O(\log_2 n)$	$O(\log_2 n)$
	경사트리	$O(n)$	$O(n)$	$O(n)$
해싱	최선의 경우	$O(1)$	$O(1)$	$O(1)$
	최악의 경우	$O(n)$	$O(n)$	$O(n)$



감사합니다!